

Data Warehousing and Data Mining

Complete MCQ-Oriented Notes

IBPS SO (IT OFFICER)
COMPLETE PREPARATION SERIES

100% SYLLABUS COVERAGE
CONCEPT-FOCUSED LEARNING
MCQ-ORIENTED APPROACH
QUICK REVISION READY

COMPILED BY
Asabhya Maanav

2026 EDITION
VERSION 1.0

Table of Contents

Data Warehousing and Data Mining - Complete Notes	4
1. Data Warehouse - Process and Technology	4
1.1 Introduction and Definition	4
1.2 OLTP vs OLAP (Operational DB vs Data Warehouse)	4
1.3 Data Warehouse Architecture (Three-Tier)	5
1.4 Warehouse Planning and Management	5
2. Warehouse Infrastructure	6
2.1 Hardware and Operating Systems	6
2.2 Client/Server Computing Model	6
2.3 Parallel Processors and Cluster Systems	6
2.4 Distributed DBMS Implementations	6
2.5 Warehousing Software	6
3. Warehouse Design (Schema, ETL, Metadata, Data Marts)	6
3.1 Multidimensional Data Model	6
3.2 Schema Design	7
3.3 Data Acquisition - ETL	7
3.4 Metadata	8
3.5 Data Marts	8
4. Data Mining - Introduction	8
4.1 Definition and Motivation	8
4.2 KDD - Knowledge Discovery in Databases	8
4.3 Data Mining Functionalities	9
4.4 Data Mining Architecture	9
4.5 Data Mining vs Data Warehousing vs OLAP	9
5. Data Processing and Preprocessing	9
5.1 Why Preprocess - Forms of Preprocessing	9
5.2 Data Cleaning	10
5.3 Data Integration and Transformation	10
5.4 Data Reduction	11
6. Data Visualization and OLAP	11
6.1 Data Visualization and Overall Perspective	11
6.2 OLAP - Functions and Operations	11
6.3 OLAP Servers - ROLAP vs MOLAP vs HOLAP	12
7. Data Mining Techniques (Core Algorithms)	12
7.1 Association Rule Mining	12
7.2 Classification (Supervised Learning)	13

7.3 Clustering (Unsupervised Learning)	14
7.4 Prediction (Regression)	14
8. Warehouse Management and Advanced Mining	15
8.1 Warehouse Management	15
8.2 Applications and Recent Trends	15
9. Quick Revision - High-Yield Exam Pointers	15

Data Warehousing and Data Mining - Complete Notes

A **Data Warehouse (DW)** is a **subject-oriented, integrated, time-variant and non-volatile** collection of data that supports management's **decision-making** process. **Data Mining** is the **non-trivial extraction of implicit, previously unknown and potentially useful** knowledge from large data stores. This guide deconstructs the entire IBPS SO (IT Officer) Data Warehousing & Data Mining syllabus to the atomic level, building from first principles to advanced mining algorithms.

Memory Trick: A data warehouse is **"SITN"** = **S**ubject-oriented, **I**ntegrated, **T**ime-variant, **N**on-volatile (Inmon's 4 keywords).

1. Data Warehouse - Process and Technology

1.1 Introduction and Definition

1.1.1 Definition of a Data Warehouse

The classic definition by **Bill Inmon (the "Father of Data Warehousing", 1990)** states: a data warehouse is a **subject-oriented, integrated, time-variant, non-volatile** collection of data in support of management's decision-making process.

- **Subject-oriented:** organized around major **subjects** (customer, product, sales, supplier) rather than around day-to-day **operations/applications**. It excludes data not useful for decision support.
- **Integrated:** built by integrating **multiple heterogeneous sources** (flat files, RDBMS, online records) with consistent **naming, encoding, units and formats**. Data-cleaning and data-integration are applied.
- **Time-variant:** every record carries a **time element**; the warehouse keeps **historical data** (typically **5-10 years**) unlike an operational DB that holds only current values.
- **Non-volatile:** data is **read-only** for end users; once loaded it is **not updated or deleted** in real time. Only two operations occur: **initial loading** and **access (query)**.

MCQ Trap: Operational DBs hold **current** data and allow updates; warehouses are **non-volatile** and **time-variant** (historical). "Volatile" warehouse is always WRONG.

1.1.2 Need / Motivation for a Data Warehouse

- Operational (OLTP) systems are tuned for **transactions**, not **analysis**.
- Decision support needs **historical, summarized, integrated** data across the enterprise.
- Avoids running heavy analytical queries on **production** systems.
- Provides a **single version of the truth** for the organization.

1.2 OLTP vs OLAP (Operational DB vs Data Warehouse)

OLTP (Online Transaction Processing) runs day-to-day operations; **OLAP (Online Analytical Processing)** runs multidimensional analysis on the warehouse.

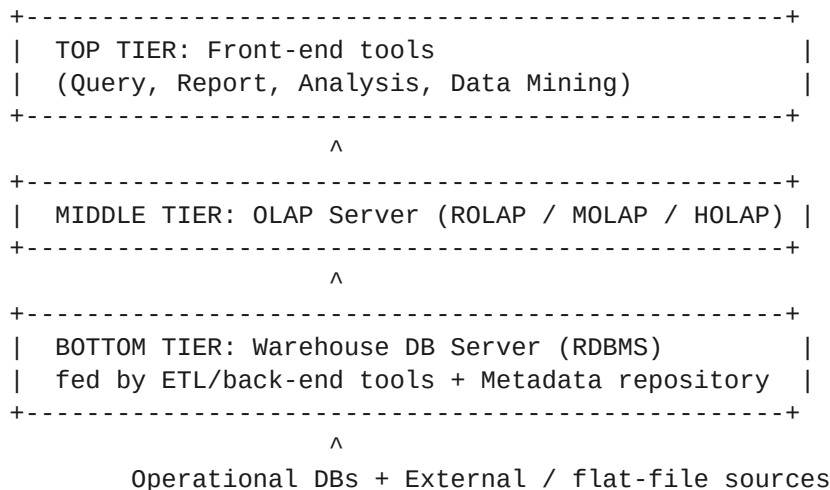
Feature	OLTP (Operational DB)	OLAP (Data Warehouse)
Purpose	Day-to-day transactions	Decision support / analysis
Orientation	Application -oriented	Subject -oriented
Data	Current , detailed	Historical , summarized
Design	Normalized (3NF) , ER model	Denormalized , star/snowflake
Access	Read/Write (INSERT/UPDATE/DELETE)	Mostly read-only queries
Unit of work	Short, simple transaction	Complex, long analytical query
Records accessed	Tens	Millions

Feature	OLTP (Operational DB)	OLAP (Data Warehouse)
Users	Clerks, thousands	Analysts, managers, hundreds
Size	MB to GB	GB to TB / PB
Metric	Transactions per second	Query throughput , response time

Memory Trick: OLTP = "T" for Transaction (current, write); OLAP = "A" for Analysis (history, read).

1.3 Data Warehouse Architecture (Three-Tier)

The standard architecture has **3 tiers**: a **bottom** data tier, a **middle** OLAP tier, and a **top** client tier.



- **Bottom tier:** the **warehouse database server** (almost always an **RDBMS**) plus **back-end ETL** tools and the **metadata repository**; also holds **data marts**.
- **Middle tier:** the **OLAP server**, implemented as **ROLAP**, **MOLAP** or **HOLAP**.
- **Top tier:** **front-end client** tools for querying, reporting, analysis and data mining.

1.4 Warehouse Planning and Management

1.4.1 Warehousing Strategy (Design Approaches)

There are **2 dominant build strategies**, named after their proponents:

Aspect	Top-Down (Inmon)	Bottom-Up (Kimball)
Build order	Enterprise DW first , then marts	Data marts first , then integrate
Data model	Normalized (3NF)	Dimensional (star)
Cost / time	High , slow to deliver	Low , fast to deliver
Risk	Higher upfront	Lower, incremental
Result	True single repository	Bus of conformed marts

MCQ Trap: Inmon = Top-Down (normalized, EDW first); Kimball = Bottom-Up (dimensional, marts first). Examiners swap the two names.

1.4.2 Warehouse Management and Support Processes

- **Data acquisition** (ETL), **data management** (storage, partitioning, archiving), and **metadata management**.
- Support processes: **scheduling**, **backup/recovery**, **security**, **monitoring**, **tuning**, **capacity planning**.

1.4.3 Warehouse Planning and Implementation

- Key steps: **requirement analysis** -> **data modeling** -> **source identification** -> **ETL design** -> **schema/cube build** -> **front-end** -> **deploy** -> **maintain**.
- Implementation pitfalls: under-estimating **data quality** effort, scope creep, weak **metadata**.

2. Warehouse Infrastructure

2.1 Hardware and Operating Systems

- DW needs **high I/O throughput**, large storage, large memory and strong parallelism.
- RAID** arrays provide redundancy and parallel I/O; **SANs** provide scalable shared storage.
- Operating systems used: **UNIX/Linux** (scalability), **Windows Server**, and proprietary appliance OSes.

2.2 Client/Server Computing Model

- DW uses the **client/server** model: clients (front-end tools) request, servers (DB/OLAP) process.
- Tiers: **2-tier** (client + DB server) and **3-tier / n-tier** (adds an application/OLAP tier for scalability).

2.3 Parallel Processors and Cluster Systems

Parallel hardware architectures for warehousing:

Architecture	Memory	Disk	Notes
SMP (Symmetric Multiprocessing)	Shared	Shared	Tightly coupled, limited scale
MPP (Massively Parallel Processing)	Private	Private	Shared-nothing , highly scalable
NUMA	Shared (non-uniform)	Shared	Local memory faster than remote
Cluster	Private per node	Shared or private	Nodes joined by fast interconnect

- Speed-up**: more processors -> same job runs **faster** (fixed size).
- Scale-up**: more processors -> handle **proportionally larger** job in same time.
- Query parallelism: **inter-query** (different queries in parallel) and **intra-query** (one query split across CPUs).

MCQ Trap: MPP = shared-nothing; SMP = shared-everything (shared memory + disk). NUMA = shared memory but **non-uniform** access time.

2.4 Distributed DBMS Implementations

- A **Distributed DBMS (DDBMS)** stores data across **multiple sites** connected by a network.
- Fragmentation**: **horizontal** (rows), **vertical** (columns), **mixed/hybrid**.
- Replication**: copies of data at several sites for availability and locality.
- Transparency types**: **location**, **fragmentation**, **replication** transparency.

2.5 Warehousing Software

- ETL tools** (Informatica, SSIS, DataStage, Talend), **RDBMS/MPP databases** (Teradata, Oracle, Netezza), **OLAP engines** (Essbase, SSAS), **BI/reporting** (Cognos, BusinessObjects, Tableau).

3. Warehouse Design (Schema, ETL, Metadata, Data Marts)

3.1 Multidimensional Data Model

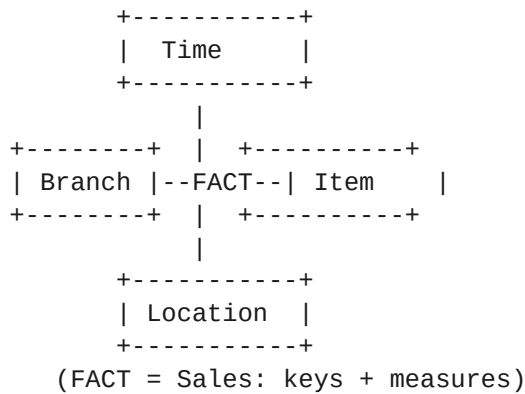
- Data is modeled as a **data cube**: **dimensions** (perspectives, e.g. time, item, location) and **measures/facts** (numeric values, e.g. sales amount).

- A cube of **n dimensions** is sometimes called a **base cuboid**; the fully aggregated cell is the **apex cuboid**; the lattice of cuboids is the **data cube**.
- The **grain** of a fact table = the **level of detail** of one row.

3.2 Schema Design

3.2.1 Star Schema

A **central fact table** is connected to a set of **denormalized dimension tables**, forming a star.



- Each dimension is a **single table** (denormalized, may have redundancy).
- **Fewer joins** -> **fast** query performance; simplest and most common schema.

3.2.2 Snowflake Schema

A refinement of the star where **dimension tables are normalized** into multiple related sub-tables.

- Splits a dimension (e.g. Item -> Item, Supplier; Location -> City, State, Country).
- **Less redundancy / less storage** but **more joins** -> **slower** queries; more complex.

3.2.3 Fact Constellation (Galaxy) Schema

- **Multiple fact tables share** common **dimension tables**.
- Used for complex applications and at the **enterprise** level; also called a **galaxy** schema.

Property	Star	Snowflake	Fact Constellation
Fact tables	One	One	Multiple
Dimensions	Denormalized	Normalized	Shared dimensions
Joins	Fewest	More	Most
Redundancy	High	Low	Mixed
Query speed	Fastest	Slower	Varies

MCQ Trap: Star = **denormalized** dimensions (1 table each); **Snowflake** = **normalized** dimensions (split tables); **Galaxy/Constellation** = **many fact tables sharing dimensions**.

3.2.4 Fact Tables and Dimension Tables

- **Fact table:** holds **numeric measures** + **foreign keys** to dimensions; very **deep** (many rows), narrow.
- **Dimension table:** holds **descriptive attributes** (textual) for filtering/grouping; **wide**, fewer rows.
- **Types of facts:** **additive** (sum over all dimensions, e.g. sales), **semi-additive** (sum over some, not time, e.g. balance), **non-additive** (cannot sum, e.g. ratios).

3.3 Data Acquisition - ETL

ETL = Extract, Transform, Load - the back-end process that populates the warehouse.

SOURCES --> [EXTRACT] --> [TRANSFORM] --> [LOAD] --> WAREHOUSE

(OLTP, files, APIs)	pull data	clean, standardize, dedupe, derive	insert /update	(DW / data marts)
---------------------------	--------------	--	-------------------	-------------------------

- **Extract:** read data from **heterogeneous source** systems (full or incremental).
- **Transform:** **clean, standardize, deduplicate, derive, aggregate, validate** against business rules (the heaviest stage).
- **Load:** write to the target via **initial load, incremental load** or **full refresh**.
- **ELT** variant (modern): **Load first, then Transform** inside a powerful target engine.

Memory Trick: ETL = "ETL" Extract -> Transform -> Load; ELT just swaps the last two (load into the warehouse, transform there).

3.4 Metadata

Metadata = "data about data" - stored in the **metadata repository**; it is essential for navigation, lineage and administration.

Type	Audience	Content
Technical metadata	Developers/DBAs	source schemas, ETL mappings , data types
Business metadata	Analysts/managers	business terms, definitions, ownership, rules
Operational metadata	Operations	data lineage , currency, ETL job logs , usage stats

3.5 Data Marts

- A **data mart** is a **subset** of the warehouse focused on a **single subject / department** (e.g. Sales mart).
- **Dependent data mart:** sourced from the **central enterprise warehouse** (top-down).
- **Independent data mart:** **standalone**, built directly from **operational sources** (bottom-up).

MCQ Trap: Dependent mart feeds FROM the DW; independent mart bypasses the DW and reads sources directly.

4. Data Mining - Introduction

4.1 Definition and Motivation

- **Data mining** = the process of discovering **interesting patterns, correlations and knowledge** from **large** amounts of data; also called **Knowledge Discovery from Data (KDD)**.
- Motivation: **"data rich but information poor"** - explosive data growth needs automated analysis.
- Patterns must be **valid, novel, useful and understandable**.

4.2 KDD - Knowledge Discovery in Databases

KDD is the overall **iterative process**; **data mining is just ONE step** in it. The classic **7 steps** (Fayyad):

```

1 Data Cleaning      \
2 Data Integration   > = Data Preprocessing
3 Data Selection     |
4 Data Transformation/
5 DATA MINING       <-- core pattern-discovery step
6 Pattern Evaluation
7 Knowledge Presentation
  
```

MCQ Trap: Data mining is a **STEP** within **KDD**, not a synonym for it. The step that finds patterns is step 5; cleaning/integration come first.

4.3 Data Mining Functionalities

These specify the **kinds of patterns** to be found, split into **descriptive** and **predictive**.

- **Descriptive mining:** characterizes **general properties** of data (clustering, association, summarization).
- **Predictive mining:** performs **inference** to **predict** unknown values (classification, regression).

Functionalities:

- **Class/Concept description: characterization** (summary of a class) and **discrimination** (comparison between classes).
- **Frequent patterns / Association / Correlation:** finds items that co-occur.
- **Classification and Regression:** predict **categorical** (classification) or **numeric** (regression) labels.
- **Cluster analysis:** group objects with **no predefined classes**.
- **Outlier analysis:** find objects that **deviate** (outliers); useful for **fraud detection**.
- **Evolution analysis:** model trends/changes over **time** (time-series).

4.4 Data Mining Architecture

The standard architecture has these components:

- **Data sources:** database, **data warehouse**, WWW, or other repositories.
- **Database / Data warehouse server:** fetches relevant data.
- **Knowledge base: domain knowledge** used to guide and evaluate search.
- **Data mining engine:** the core set of functional modules (association, classification, clustering...).
- **Pattern evaluation module:** uses **interestingness measures** to filter patterns.
- **User interface (GUI):** lets users specify queries and visualize results.

4.5 Data Mining vs Data Warehousing vs OLAP

Aspect	Data Warehousing	OLAP	Data Mining
Goal	Store integrated history	Summarize / navigate	Discover patterns
Operation	ETL + storage	Roll-up, drill-down, slice	Apriori, k-means, trees
Style	Repository	Multidimensional analysis	Knowledge discovery
Query type	Known	Known, interactive	Unknown, automatic

MCQ Trap: OLAP answers **"what happened"** (verification-driven); data mining answers **"why / what will happen"** and is **discovery-driven**.

5. Data Processing and Preprocessing

5.1 Why Preprocess - Forms of Preprocessing

Real-world data is **dirty: incomplete, noisy and inconsistent**. Quality data -> quality patterns ("garbage in, garbage out"). The **4 (5) major forms**:

- **Data cleaning** - fill missing values, smooth noise, fix inconsistencies.
- **Data integration** - merge multiple sources.
- **Data transformation** - normalize, aggregate, generalize.
- **Data reduction** - shrink volume while keeping analytical quality.
- **Discretization** - special reduction for numeric attributes (often listed under reduction).

Memory Trick: Preprocessing = **"CITRD"** = **C**leaning, **I**ntegration, **T**ransformation, **R**eduction, **D**iscretization.

5.2 Data Cleaning

5.2.1 Handling Missing Values

Methods (in order of increasing quality):

- **Ignore the tuple** (only if class label missing / many attributes missing).
- **Fill manually** (tedious, infeasible for big data).
- **Use a global constant** like "Unknown" or **-infinity**.
- **Use the attribute mean / median** for all samples.
- **Use the attribute mean for the same class** (smarter).
- **Use the most probable value** (predict via **regression**, **Bayesian** inference, or **decision tree**) - the best.

5.2.2 Handling Noisy Data

Noise = random error or variance in a measured variable. Smoothing techniques:

- **Binning: sort** values, partition into **bins** (equal-width or equal-frequency), then smooth by **bin means, bin medians or bin boundaries**.
- **Regression:** fit data to a **regression function** (linear / multiple) to smooth.
- **Clustering:** group values; points outside clusters are **outliers** to remove.
- **Combined computer and human inspection:** machine flags suspects, humans verify.

Binning example (sort: 4, 8, 15, 21, 21, 24, 25, 28, 34)

Equal-frequency bins (depth 3):

Bin1: 4, 8, 15 -> mean smoothing -> 9, 9, 9

Bin2: 21, 21, 24 -> mean smoothing -> 22, 22, 22

Bin3: 25, 28, 34 -> mean smoothing -> 29, 29, 29

MCQ Trap: Binning, regression and clustering are the standard **noise-smoothing** trio. Binning smooths using **neighborhood** (sorted local values).

5.2.3 Handling Inconsistent Data

- Use **external references**, **domain knowledge** and **functional dependencies** to correct discrepancies (e.g. conflicting codes, formats).

5.3 Data Integration and Transformation

5.3.1 Data Integration Issues

- **Schema integration / Entity identification problem:** match equivalent entities (e.g. **cust_id** vs **customer_no**).
- **Redundancy:** an attribute derivable from another; detect with **correlation analysis - chi-square** for nominal, **correlation coefficient (Pearson)** for numeric data.
- **Tuple duplication** and **data value conflicts** (different units, scales, encodings).

5.3.2 Data Transformation

- **Smoothing** (remove noise), **Aggregation** (summarize), **Generalization** (climb concept hierarchy), **Attribute construction**, and **Normalization**.

Normalization methods (scale values to a small range):

- **Min-max:** maps to $[new_min, new_max]$. Formula: $v' = (v - min) / (max - min) * (new_max - new_min) + new_min$.
- **Z-score (zero-mean):** $v' = (v - mean) / std_dev$; used when min/max unknown or outliers present.
- **Decimal scaling:** $v' = v / 10^j$, where **j** is the smallest integer with $max(|v'|) < 1$.

5.4 Data Reduction

Obtain a **reduced representation** that yields the **same (or almost same) analytical results** with much less volume.

Technique	Idea
Data cube aggregation	Aggregate to higher levels (e.g. daily -> yearly)
Dimensionality reduction	Remove/encode irrelevant attributes
Numerosity reduction	Replace data with a smaller model/sample
Data compression	Encode data into compressed form
Discretization	Convert continuous values into intervals

5.4.1 Dimensionality Reduction

- Reduces the number of **attributes (columns)**.
- Methods: **Wavelet transforms (DWT)**, **Principal Component Analysis (PCA / Karhunen-Loeve)**, and **attribute subset selection** (forward selection, backward elimination, decision-tree induction).

5.4.2 Numerosity Reduction

- Reduces the number of **data values (rows)**.
- **Parametric**: store only **model parameters** - **regression** and **log-linear** models.
- **Non-parametric**: **histograms, clustering, sampling** (SRSWOR, SRSWR, stratified), **data cube aggregation**.

5.4.3 Data Compression

- **Lossless**: original data fully recoverable (e.g. string/run-length encoding).
- **Lossy**: only an approximation recovered (e.g. **DWT, PCA**).

5.4.4 Discretization and Concept Hierarchy Generation

- **Discretization** divides a continuous attribute's range into **intervals**, replacing raw values with **interval labels**.
- **Supervised** (uses class labels, e.g. **entropy-based**) vs **unsupervised** (binning, clustering).
- **Top-down / splitting** (e.g. entropy-based) vs **bottom-up / merging** (e.g. **ChiMerge**).
- **Concept hierarchy** maps **low-level** to **higher-level** concepts, e.g. **street < city < state < country**; for numeric data use the **3-4-5 rule** for intuitive partitioning.

MCQ Trap: Dimensionality reduction removes ATTRIBUTES (columns); numerosity reduction reduces the number of ROWS/values. Don't swap them.

6. Data Visualization and OLAP

6.1 Data Visualization and Overall Perspective

- Conveys patterns visually; supports the **3 warehouse essentials**: **aggregation** (summary at many levels), **historical information** (time-variant trends), and a powerful **query facility** (ad-hoc analysis).

6.2 OLAP - Functions and Operations

OLAP = Online Analytical Processing - fast, multidimensional, interactive analysis of warehouse data. **E. F. Codd (1993)** defined **12 rules** of OLAP; the alternative test is **FASMI** (**F**ast **A**nalysis of **S**hared **M**ultidimensional **I**nformation).

The **5 core OLAP operations** on the data cube:

- **Roll-up (drill-up): aggregate** by climbing a concept hierarchy **or** by **dimension reduction** (e.g. city -> country).
- **Drill-down (roll-down):** reverse of roll-up - go to **finer detail** (e.g. quarter -> month) or **add a dimension**.
- **Slice:** select on **ONE** dimension -> produces a **sub-cube** (e.g. time = "Q1").
- **Dice:** select on **TWO or more** dimensions -> a smaller sub-cube.
- **Pivot (rotate): reorient** the cube axes for visualization (e.g. swap rows and columns).

Roll-up : Month --> Quarter --> Year (less detail, aggregate)
 Drill-down: Year --> Quarter --> Month (more detail)
 Slice : pick 1 dimension value (a plane)
 Dice : pick >=2 dimension ranges (a box)
 Pivot : rotate axes (rows <-> columns)

MCQ Trap: Slice = ONE dimension, Dice = TWO or more dimensions. Roll-up aggregates (up), drill-down dis-aggregates (down).

6.3 OLAP Servers - ROLAP vs MOLAP vs HOLAP

Feature	ROLAP	MOLAP	HOLAP
Storage	Relational tables	Multidimensional array (cube)	Hybrid (both)
Detail data	In RDBMS	In cube	In relational
Aggregations	Computed via SQL	Pre-computed in cube	In MOLAP cube
Data volume	Very large (scalable)	Limited (cube size)	Large
Speed	Slower (SQL joins)	Fastest for aggregates	Balanced
Issue	Many joins	Sparse-matrix explosion	Complexity
Example	MicroStrategy	Essbase	MS SSAS

- **ROLAP:** stores data in **relational** star/snowflake tables; computes aggregates **on the fly** with SQL; best for **huge** data.
- **MOLAP:** stores data in **pre-aggregated multidimensional cubes**; **fastest** query response; suffers from **data sparsity / cube explosion**.
- **HOLAP: hybrid** - keeps **detailed data in relational** stores and **aggregations in MOLAP** cubes, balancing scalability and speed.

Memory Trick: ROLAP = Relational; MOLAP = Multidimensional (cube, fast); HOLAP = Hybrid (best of both).

7. Data Mining Techniques (Core Algorithms)

7.1 Association Rule Mining

Finds rules of the form **A => B** ("market-basket analysis"). The key measures:

- **Support(A=>B) = P(A U B)** = fraction of transactions containing **both** A and B.
- **Confidence(A=>B) = P(B|A) = Support(A U B) / Support(A)** = reliability of the rule.
- **Lift(A=>B) = Confidence(A=>B) / Support(B) = Support(A U B) / (Support(A) x Support(B)).**
- **Lift = 1** -> A and B **independent**; **Lift > 1** -> **positively** correlated; **Lift < 1** -> **negatively** correlated.
- A **frequent itemset** has **support >= min_support**; a **strong rule** satisfies **both min_support** and **min_confidence**.

Tx | Items
 1 | {Milk, Bread, Butter}
 2 | {Bread, Butter}

```

3 | {Milk, Bread}
4 | {Milk, Bread, Butter}
Support(Bread, Butter) = 3/4 = 75%
Confidence(Bread=>Butter)=Sup(B, Bu)/Sup(Bread)=3/4 / 4/4 = 75%

```

7.1.1 Apriori Algorithm

Proposed by **Agrawal and Srikant (1994)**; a **level-wise**, breadth-first algorithm.

- **Apriori property (anti-monotone): every non-empty subset of a frequent itemset is also frequent**; equivalently, **if an itemset is infrequent, all its supersets are infrequent** (used to prune).
- **2 steps per level: Join** (generate candidates **C_k** from **L_{k-1}**) and **Prune** (remove candidates with an infrequent subset).
- Scans the database **once per level** (multiple passes).
- **Drawbacks: many database scans** and a potentially **huge candidate set**.

7.1.2 FP-Growth (Frequent Pattern Growth)

Proposed by **Han, Pei, Yin (2000)**; uses a compact **FP-tree** with **no candidate generation**.

- Requires only **2 database scans**: 1st counts **frequent 1-itemsets** (and their order), 2nd builds the **FP-tree**.
- Uses **divide-and-conquer** via **conditional pattern bases** and **conditional FP-trees**.
- Generally **much faster** than Apriori on dense data.

MCQ Trap: Apriori = candidate generation + many scans; FP-Growth = NO candidate generation + only 2 scans (FP-tree).

7.2 Classification (Supervised Learning)

Predicts **categorical class labels**; a **2-step** process: **(1) model construction** from a labeled **training set**, **(2) model usage** to classify new (test) data after accuracy estimation.

7.2.1 Decision Tree Induction

A **top-down, recursive, divide-and-conquer** tree where internal nodes test attributes and leaves give classes.

Algorithm	Author	Split measure	Notes
ID3	Quinlan	Information Gain (entropy)	categorical only
C4.5	Quinlan	Gain Ratio	handles continuous + missing
CART	Breiman	Gini Index	strictly binary trees

- **Entropy / Info(D) = - SUM p_i log₂(p_i)** (impurity of dataset D).
- **Information Gain = Info(D) - Info_A(D)**; ID3 picks the attribute with the **highest gain**.
- **Gini Index = 1 - SUM (p_i)²**; CART picks the **lowest Gini** split.
- **Overfitting** is controlled by **pruning** (pre-pruning or post-pruning).

MCQ Trap: ID3 -> Information Gain, C4.5 -> Gain Ratio, CART -> Gini Index. ID3 is biased toward attributes with many values; gain ratio fixes this.

7.2.2 Bayesian Classification (Naive Bayes)

Based on **Bayes' Theorem**:

$$P(H | X) = \frac{P(X | H) * P(H)}{P(X)}$$

$P(H|X)$ =posterior, $P(H)$ =prior,
 $P(X|H)$ =likelihood, $P(X)$ =evidence

- **Naive Bayes** assumes **class-conditional independence** (attributes independent given the class) - "naive" but fast and accurate in practice.

7.2.3 k-Nearest Neighbour (k-NN)

- A **lazy / instance-based** learner: stores all training data and classifies a new point by **majority vote of its k nearest neighbours** using a **distance** measure (usually **Euclidean**).
- **Lazy learner** (k-NN) does work at **classification** time; an **eager learner** (decision tree, Naive Bayes) builds a model at **training** time.

7.2.4 Neural Networks (Overview)

- Layers of interconnected **neurons** trained via **backpropagation**; good accuracy, but a **black box** (poor interpretability) and slow to train.

7.3 Clustering (Unsupervised Learning)

Groups objects so that intra-cluster similarity is **high** and inter-cluster similarity is **low**; there are **no predefined class labels**.

7.3.1 Partitioning Methods

- **k-Means: centroid-based**; pick **k**, assign each point to the **nearest centroid (mean)**, recompute centroids, repeat until stable. Complexity **$O(n k t)$** (n points, k clusters, t iterations). Fast but **sensitive to outliers/noise**, needs **k** in advance, and finds only **spherical/convex** clusters.
- **k-Medoids (PAM)**: uses an **actual representative object (medoid)** as the center instead of the mean; **more robust to outliers** but **costlier** (about **$O(k(n-k)^2)$** per iteration).

k-Means loop:

- 1 choose k initial centroids
- 2 assign each point -> nearest centroid
- 3 recompute centroid = mean of its points
- 4 repeat 2-3 until no change (convergence)

7.3.2 Hierarchical Methods

- **Agglomerative (AGNES): bottom-up** - start with each point as its own cluster and **merge** the closest pair repeatedly.
- **Divisive (DIANA): top-down** - start with one big cluster and **split** repeatedly.
- Result visualized as a **dendrogram**; linkage = single, complete, average.

7.3.3 Density-Based Methods

- **DBSCAN**: groups dense regions using **Eps** (radius) and **MinPts** (minimum points); finds **arbitrary-shaped** clusters and labels low-density points as **noise/outliers**; does **not** need k.
- Others: **OPTICS**, **DENCLUE** (density), **STING**, **CLIQUE** (grid).

MCQ Trap: Classification = supervised (labeled); Clustering = unsupervised (no labels). k-Means uses MEAN (sensitive to outliers); k-Medoids uses a real object (robust).

7.4 Prediction (Regression)

- **Regression** predicts a **continuous numeric** value (vs classification's discrete label).
- **Linear regression: $y = w \cdot x + b$** (fit a straight line by least squares); **multiple linear regression** uses several predictors; **non-linear regression** for curved relationships.

Aspect	Classification	Clustering	Regression/Prediction
Learning	Supervised	Unsupervised	Supervised
Output	Discrete class	Groups	Continuous value
Labels needed	Yes	No	Yes

8. Warehouse Management and Advanced Mining

8.1 Warehouse Management

- **Data mining interface:** front-end through which users specify mining tasks and view patterns.
- **Security: authentication, authorization (roles/GRANT), encryption, auditing;** row/column-level access for sensitive data.
- **Backup and Recovery: full / incremental backups,** logs and **checkpoints** to restore after failure.
- **Tuning the warehouse: indexing (bitmap, B-tree, join indexes), partitioning, materialized views, query rewrite, aggregate pre-computation.**
- **Testing the warehouse: ETL/data-quality testing,** reconciliation of source-to-target counts, performance and regression testing.

Memory Trick: Bitmap indexes shine on **low-cardinality** dimension columns (gender, flags); B-tree indexes suit **high-cardinality** keys.

8.2 Applications and Recent Trends

8.2.1 Types of Warehousing Applications

- **Information processing** (query/report), **Analytical processing** (OLAP), and **Data mining** (knowledge discovery).
- Domains: **retail/market-basket, banking/fraud, telecom, healthcare, CRM, finance.**

8.2.2 Web Mining

Mining the **WWW**; it has **3 categories**:

- **Web content mining:** mine the **content** of web pages (text, images).
- **Web structure mining:** mine the **hyperlink structure** - algorithms **PageRank** (Google) and **HITS** (hubs & authorities).
- **Web usage mining:** mine **server logs / clickstream** to discover **user access patterns**.

MCQ Trap: Web mining's three types are **Content, Structure, Usage**. **PageRank/HITS** belong to **structure** mining (link analysis).

8.2.3 Spatial Mining

- Mining **spatial / geographic (GIS)** data; finds **spatial association rules, spatial clusters and co-location** patterns (e.g. "houses near a lake cost more").

8.2.4 Temporal Mining

- Mining **time-stamped** data; includes **time-series analysis, sequential-pattern mining** and **trend/periodicity** discovery.

8.2.5 Text and Multimedia Mining (Overview)

- **Text mining:** extracts knowledge from **unstructured text** (uses Information Retrieval + NLP); handles keyword/term-frequency, classification, clustering of documents.
- **Multimedia mining:** mines **images, audio and video** (content-based retrieval, pattern recognition).

9. Quick Revision - High-Yield Exam Pointers

- **Inmon's 4 keywords:** subject-oriented, integrated, time-variant, non-volatile.
- **OLTP** = current + write + normalized; **OLAP** = historical + read + denormalized.
- **Star** = denormalized 1-table dimensions; **Snowflake** = normalized; **Galaxy** = many fact tables.
- **ETL** = Extract -> Transform -> Load; **ELT** loads first.
- **KDD** has **7 steps**; **data mining is step 5** (one step only).

- **Normalization:** min-max, z-score, decimal scaling.
- **OLAP ops:** roll-up, drill-down, **slice (1 dim)**, **dice (≥ 2 dims)**, pivot.
- **ROLAP** relational, **MOLAP** cube (fast), **HOLAP** hybrid.
- **Support, Confidence, Lift; Apriori** (candidates, many scans) vs **FP-Growth** (no candidates, 2 scans).
- **ID3=Info Gain, C4.5=Gain Ratio, CART=Gini.**
- **k-Means** mean (outlier-sensitive) vs **k-Medoids** medoid (robust).
- **Classification = supervised; Clustering = unsupervised.**
- **Web mining** = content, structure, usage.